OPCP AI-Powered Store - Marketing Document

Generated on 2026-06-16

OPCP AI-Powered Store

Objective

OPCP AI-Powered Store is a centralized application deployment and management platform designed for GenAI agents. It provides automated deployment, lifecycle management, and SSO authentication for web applications through multiple interfaces (Web, CLI, API, MCP).

Core Purpose: Enable GenAI agents to autonomously deploy, manage, and access web applications without human intervention.

Features

- User registration and authentication with Gitea integration
- Web-based dashboard with multi-language support (EN/FR)
- Application management with **PostgreSQL database** (enterprise-grade performance)
- SSO Identity Provider with token-based authentication
- **Application deployment system** (Clone, Start, Stop, Monitor, Logs)
- Docker containerization with docker-compose
- Command-line interface (CLI) with comprehensive commands
- REST API endpoints with streaming support (Server-Sent Events)
- MCP (Model Context Protocol) support
- ModSecurity WAF protection with OWASP CRS rules
- Automated database backups with PostgreSQL pg_dump
- Billing and cost tracking system with activity logging
- **Virtual AI Agents**: AI Chat Developer and Operations assistants
- Database health monitoring and statistics
- Multi-server deployment support with capacity management
- **Dynamic Nginx Locations**: Automatic reverse proxy configuration per user/app
- Comprehensive user guide and documentation
- **PostgreSQL connection pooling** with thread-safe operations
- **SQLite to PostgreSQL migration tools**
- **MIG Shared GPU**: NVIDIA Multi-Instance GPU partitioning and management per server
- **Serverless Docker Execution**: Submit and run Docker-based jobs on-demand
- **Multi-server Replication**: Peer-to-peer database replication with sync tokens
- **App Orchestrator**: Automated application lifecycle orchestration with reconciliation
- **Password Reset & 2FA**: Secure password recovery and two-factor authentication via email

PostgreSQL Migration

Database Migration ```bash # Run the automated migration from SQLite to PostgreSQL python3 ./migration

Configure PostgreSQL connection (environment variables) export POSTGRES_HOST

*Migration Features - **Complete Data Migration**: Preserves all existing data during transition - **Type C*

OPCP AI-Powered Store - Marketing Document

Generated on 2026-06-16

Installation Methods

Prerequisites Check ```bash # Verify system requirements which python3 && which pip && which docker &

Interactive Deployment (Recommended) ```bash # Clone repository git clone https://github.com/your-repo/c

Interactive deployment with menu selection ./deployControlPlan.sh start ```

Local System Deployment ```bash # Deploy directly on host system (production) ./deployControlPlan.sh sta

With custom user parameters ./deployControlPlan.sh start locally 123 "John Doe" "jo

Docker Deployment ```bash # Deploy using Docker containers (development/testing) ./deployControlPlan.s

Or direct docker-compose docker-compose up -d --build ```

Manual Installation Steps ```bash # 1. Install Python dependencies pip install -r requirements.txt

2. Initialize database python3 ./scripts/aipoweredstore_cli.py init-db

3. Start application python3 src/ControlPlanFlaskApp_postgres.py ```

Bare-Metal Installation (Ubuntu/Debian)

For a fresh bare-metal server running Ubuntu 22.04+ or Debian 12+, the `init_pltf.sh` script automates the complete platform provisioning. This is the recommended method for production deployments on OVHcloud dedicated servers or VPS.

Prerequisites

Requirement	Details
OS	Ubuntu 22.04+ or Debian 12+
User	Non-root user with `sudo` privileges
Network	Internet access (public interface)
SSH key	Configured for GitHub repository access
GPU (optional)	NVIDIA H100, A100, or A30 for MIG shared GPU

Run the provisioning script

```
#### What the script installs (in order)
| Step | Component | Purpose |
```

```
|------|-----------|---------|
| 1 | Python 3, pip, venv, net-tools, unzip | Application runtime and system utilities |
| 2 | Amazon Kiro CLI | AI-assisted development CLI |
| 3 | OVH shai CLI | OVHcloud infrastructure management |
| 4 | AWS CLI v2 | S3-compatible object storage access |
| 5 | Terraform 1.14.5 | Infrastructure as Code |
| 6 | Network configuration | Netplan route metrics (public: 50, private: 200) |
| 7 | Docker + docker-compose | Container orchestration engine |
| 8 | NVIDIA Driver 550 + MIG mode | GPU compute and Multi-Instance GPU partitioning |
| 9 | NVIDIA Container Toolkit | Docker GPU passthrough (`--gpus` flag) |
| 10 | AWS credentials | S3 endpoint configuration for OVHcloud |
| 11 | Repository clone | Source code + submodules |
| 12 | Python virtualenv | Dependencies from requirements.txt |
| 13 | Final setup | logs/ directory, ModSecurity config, deployments/ |
#### GPU setup detail
The script automatically handles GPU provisioning with graceful fallback:
```
NVIDIA driver install
     Success Enable MIG mode (nvidia-smi -mig 1)
                    Success Install nvidia-container-toolkit
                                      Verify Docker GPU access (30s timeout)
                    Warning GPU may not support MIG, continue
     Warning No GPU detected, skip GPU setup entirely
```
All GPU steps are **non-blocking**  the platform works without GPU hardware.
#### Post-installation (manual steps)
After `init_pltf.sh` completes, configure these items:
```bash
# 1. Platform identity
vim ~/opcp-ai-powered-store/conf/deploy.ini
# Set: DOMAIN=yourdomain.com
# Set: PLTF_NAME=Your Platform Name
# 2. SSL certificates
cp fullchain.crt ~/opcp-ai-powered-store/ssl/fullchain_domain.crt
cp private.key ~/opcp-ai-powered-store/ssl/privateKey_domain.key
# 3. S3 credentials for backups
vim ~/.aws/credentials
# Replace XXX/YYY with your OVHcloud S3 access/secret keys
# 4. Apply Docker group (required for docker commands without sudo)
newgrp docker
# Or log out and back in
# 5. Start the platform
cd ~/opcp-ai-powered-store
source .venv/bin/activate
./deployControlPlan.sh start
```
Configuration
```

Environment Variables
```bash
# Automatically generated during deployment
SECRET_KEY="auto-generated-32-byte-hex"
FLASK_ENV="production"
```
Configuration File
```bash
# Edit deployment configuration
vim ./conf/deploy.ini
# Key settings:
DOMAIN="www.swautomorph.com"
EMAIL="admin@swautomorph.com"
GITEA_VERSION="1.21.3"
MODSECURITY_CONF_DIR="/etc/nginx/modsec"
```
Database Initialization
```bash
# Initialize database schema
python3 ./scripts/aipoweredstore_cli.py init-db
# Check database health
python3 ./scripts/aipoweredstore_cli.py db-health
```
SSL Certificate Setup
```bash
# Auto-generate self-signed certificate
./scripts/generate_ssl.sh
# Or use production certificates (place in ssl/ directory)
# - fullchain_domain.crt
# - privateKey_domain.key
```
API Access for GenAI Agents
User Registration
```bash
curl -X POST https://www.swautomorph.com/register \
  -H "Content-Type: application/json" \
                                                                                -d
'{"username":"agent","email":"agent@example.com","password":"secure_pass","first_name":"
AI","last_name":"Agent"}'
```
Application Management
```bash
# List applications
curl https://www.swautomorph.com/api/applications
# Add application (admin required)
curl -X POST https://www.swautomorph.com/api/applications \
  -H "Content-Type: application/json" \
```

```
  -H "Cookie: session=your-session-cookie" \
                                    -d                  '{"name":"MyApp","description":"My
Application","git_url":"https://github.com/user/myapp.git"}'
# Deploy application with streaming
curl -X POST https://www.swautomorph.com/api/deployments \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
                                                                                    -d
'{"application_name":"MyApp","action":"clone","git_url":"https://github.com/user/myapp.g
it","server_id":1,"stream":true}'
# Application lifecycle management
curl -X POST https://www.swautomorph.com/api/deployments \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
  -d '{"application_name":"MyApp","action":"start"}'
```

Enhanced Server Management

```bash
# List servers with capacity information
curl https://www.swautomorph.com/api/servers
# Allocate server for deployment (automatic capacity-based selection)
curl -X POST https://www.swautomorph.com/api/server/allocate \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
  -d '{"application_name":"MyApp"}'
# Add new server (admin required)
curl -X POST https://www.swautomorph.com/api/servers \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
                                                                                    -d
'{"SERVER_IP":"192.168.1.100","SERVER_NAME":"worker-01","SERVER_CAPACITY_USER_MAX":20,"S
ERVER_CAPACITY_APPLI_MAX":100,"SERVER_STATUS":"STAND_BY","SERVER_TYPE":"worker"}'
```

MIG Shared GPU Management

```bash
# Enable shared GPU on a server (admin required)
curl -X PUT https://www.swautomorph.com/api/servers/1/gpu/enabled \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
  -d '{"enabled": true}'
# List available MIG profiles from GPU hardware
curl https://www.swautomorph.com/api/servers/1/gpu/profiles \
  -H "Cookie: session=your-session-cookie"
# Create MIG instances (1-7 profile IDs)
curl -X POST https://www.swautomorph.com/api/servers/1/gpu/instances \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
```

```
  -d '{"profile_ids": ["9", "14", "9"]}'
# List active MIG instances
curl https://www.swautomorph.com/api/servers/1/gpu/instances \
  -H "Cookie: session=your-session-cookie"
# Destroy all MIG instances on a server
curl -X DELETE https://www.swautomorph.com/api/servers/1/gpu/instances \
  -H "Cookie: session=your-session-cookie"
```

Serverless Docker Execution

```bash
# Submit a serverless Docker job
curl -X POST https://www.swautomorph.com/api/jobs \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
  -d '{"image": "python:3.11", "command": "python -c \"print(hello)\"", "timeout": 60}'
# Check job status
curl https://www.swautomorph.com/api/jobs/<job_id> \
  -H "Cookie: session=your-session-cookie"
# List user jobs
curl https://www.swautomorph.com/api/jobs \
  -H "Cookie: session=your-session-cookie"
```

Dynamic Nginx Locations

```bash
# Access user applications via dynamic URLs
# Format: https://www.swautomorph.com/{USER_ID}/{APPLICATION_NAME}
# Example: User 2's ai-staticwebsite running on port 6217
curl https://www.swautomorph.com/2/ai-staticwebsite
# Sync all nginx locations from database (admin required)
curl -X POST https://www.swautomorph.com/api/nginx/sync \
  -H "Cookie: session=your-session-cookie"
# Or via CLI
python3 ./scripts/sync_nginx_locations.py
```

Virtual AI Agents Integration

```bash
# AI Chat Developer Agent (code modifications)
curl -X POST https://www.swautomorph.com/api/request_dev_ai_for_app \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
          -d     '{"message":"Add     a     new     API     endpoint     for     user
management","application_name":"MyApp","application_folder":"/path/to/app","action_opera
tion":"MODIFY_CODE"}'
# AI Chat Operations Agent (deployment operations)
curl -X POST https://www.swautomorph.com/api/request_ops_ai_for_app \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
```

```
                        -d            '{"message":"[START]            Start            the
application","application_name":"MyApp","application_folder":"/path/to/app","action_oper
ation":"START"}'
# Streaming deployment with real-time logs
curl -X POST https://www.swautomorph.com/api/deployments \
  -H "Content-Type: application/json" \
  -H "Cookie: session=your-session-cookie" \
  -d '{"application_name":"MyApp","action":"start","stream":true}'
```

Enhanced CLI Interface

```bash
# Register user
python3    ./scripts/aipoweredstore_cli.py    register    --username    agent    --email
agent@example.com --password secure_pass
# List applications
python3 ./scripts/aipoweredstore_cli.py list-apps
# Add application
python3  ./scripts/aipoweredstore_cli.py  add-app  --name  MyApp  --url  https://myapp.com
--description "My Application"
# Validate SSO token
python3 ./scripts/aipoweredstore_cli.py validate-token --token your-sso-token
# Database health check with detailed statistics
python3 ./scripts/aipoweredstore_cli.py db-health
# Mount S3 storage for backups
python3 ./scripts/aipoweredstore_cli.py mount-s3fs softfluid /mnt/s3
# Initialize database with thread-safe operations
python3 ./scripts/aipoweredstore_cli.py init-db
```

MCP Protocol

```bash
# Start MCP server for agent communication
python3 ./scripts/mcp_server.py
```

Service Management

Service Status

```bash
# Check all services status
./deployControlPlan.sh ps
# View service logs
./deployControlPlan.sh logs
# Restart services
./deployControlPlan.sh restart
# Stop services
./deployControlPlan.sh stop
```

Enhanced Health Checks

```bash
```

```
# API health check
curl https://www.swautomorph.com/api/auth/status
# Database health check with statistics (admin required)
curl https://www.swautomorph.com/api/health/database
# Check Docker services
docker-compose ps
# Check deployment logs with streaming
curl https://www.swautomorph.com/api/deployments/1/logs
```

Database Management

```bash
# Create manual backup
./deployControlPlan.sh backup_db
# Recover from backup
./deployControlPlan.sh --recover_db
# Database health check
python3 ./scripts/aipoweredstore_cli.py db-health
```

Default Configuration

- **Web Interface**: https://www.swautomorph.com (or https://localhost)
- **API Endpoint**: https://www.swautomorph.com/api
- **Gitea Server**: https://www.swautomorph.com/gitea (port 3000)
- **MCP Server**: Available via scripts/mcp_server.py
- **Database**: **PostgreSQL with connection pooling** (enterprise-grade performance and scalability)
- **Deployment Directory**: /home/ubuntu/deployments/[username]/[appname]
- **SSL Certificates**: ssl/ directory
- **Logs**: logs/ directory with daily rotation and Gunicorn logging
- **Backups**: softfluid/db/backup/ with S3 sync and hourly automated backups
- **Virtual Agents**: AI Chat Developer and Operations with context-aware prompts

Architecture

Directory Structure

```
opcp-ai-powered-store/
 src/                    # Main application source
    routes/            # Flask route blueprints
       main_routes.py        # Dashboard & documentation
       auth_routes.py        # User authentication
       sso_routes.py         # Single Sign-On
       api_routes.py         # REST API with streaming
       genai_routes.py       # Virtual AI agents
       billing_routes.py     # Billing & cost tracking
       orchestrator_routes.py # App lifecycle orchestration
       replication_routes.py # Multi-server replication
       security_routes.py   # Password reset & 2FA
       serverless_routes.py  # Serverless Docker execution
       gpu_routes.py         # MIG shared GPU management
```

```
   serverless/        # Serverless execution engine
   ControlPlanFlaskApp_postgres.py    # Main Flask application
   database_postgres.py      # PostgreSQL database manager with connection pooling
   database.py               # Legacy SQLite database manager (migration compatibility)
   nginx_manager.py          # Dynamic nginx location management
   orchestrator.py           # Application orchestration & reconciliation
   replication_manager.py    # Peer-to-peer database replication
   platform_discovery.py     # Platform capability discovery
   config.py                 # Configuration & multi-language
   auth.py                   # Authentication utilities
migration/             # Database migration scripts
   add_serverless_jobs.sql          # Serverless jobs schema
   add_mig_gpu.sql                  # MIG GPU tables & server flag
   add_password_reset_and_2fa.sql   # Security features schema
   ...                              # Other migrations
scripts/               # CLI tools and utilities
   aipoweredstore_cli.py            # Command-line interface
   mcp_server.py             # Model Context Protocol server
   sync_nginx_locations.py   # Sync nginx locations from database
   postgresql_schema.sql     # PostgreSQL schema definition
tests/                 # Test suite (pytest)
   test_gpu_parsers.py              # MIG instance parser tests
   test_parse_mig_profiles.py       # MIG profile parser tests
   test_validate_profile_ids.py     # Profile ID validation tests
   test_gpu_enabled_endpoint.py     # GPU enabled toggle tests
   test_gpu_delete_instances.py     # GPU instance destruction tests
   test_serverless_routes.py        # Serverless API tests
   test_container_runtime.py        # Container runtime tests
   test_worker.py                   # Worker process tests
templates/             # HTML templates with EN/FR support
   shared_gpu.html           # MIG GPU management page
   dashboard.html            # Main dashboard
   ...                       # Other templates
static/                # CSS, JS, and static files
ssl/                   # SSL certificates
logs/                  # Application logs with Gunicorn support
shared/                # Context files for virtual agents
docs/                  # Comprehensive documentation
   USER_GUIDE.md             # AI agent usage guide
   ARCHITECTURE_GUIDE.md     # System architecture
   DEPLOYMENT_GUIDE.md       # Deployment procedures
   DATABASE_IMPROVEMENTS.md  # Database enhancements
   NGINX_DYNAMIC_LOCATIONS.md # Dynamic nginx locations guide
   VIRTUAL_AGENTS_API.md     # Virtual agents API reference
conf/                  # Configuration files
init_pltf.sh           # Platform initialization (Docker, NVIDIA drivers, MIG)
deployControlPlan.sh   # Main deployment script
```

```
### Key Components
- **Flask Application**: Multi-blueprint architecture with modular routes and virtual AI agents
- **Database**: **PostgreSQL with connection pooling** for enterprise-grade performance and thread-safe operations
- **Authentication**: Session-based with SSO token support, password reset, and two-factor authentication
- **Deployment**: Multi-server support with capacity management, automatic allocation, and streaming APIs
- **App Orchestrator**: Automated application lifecycle management with reconciliation loop
- **Serverless Execution**: Docker-based job submission and execution engine with worker processes
- **MIG Shared GPU**: NVIDIA Multi-Instance GPU partitioning via SSH with per-server configuration and web UI
- **Replication**: Peer-to-peer database replication across multiple servers with sync tokens
- **Nginx Proxy**: Dynamic location blocks for user applications with automatic configuration
- **Security**: ModSecurity WAF with OWASP CRS rules, password reset, and 2FA via email
- **Monitoring**: Health checks, database statistics, real-time streaming logs, and performance metrics
- **Virtual AI Agents**: AI Chat Developer and Operations assistants with context-aware prompts
- **Billing System**: Comprehensive cost tracking with activity logging, usage monitoring, and automated invoicing
- **Multi-language**: English/French support with session-based language switching and bilingual documentation
- **Backup System**: Automated hourly backups with S3 sync and interactive recovery tools
- **Platform Init**: Automated server provisioning including Docker, NVIDIA drivers, and MIG mode setup
## Troubleshooting
### Common Issues
```bash
# Check service status
./deployControlPlan.sh ps
# View detailed logs
./deployControlPlan.sh logs
# Port conflicts
sudo netstat -tulpn | grep -E ':(80|443|3000|5000)'
# Permission issues
sudo chown -R ubuntu:ubuntu /home/ubuntu/deployments/
sudo chown -R ubuntu:ubuntu /home/ubuntu/opcp-ai-powered-store/
# Database issues
python3 ./scripts/aipoweredstore_cli.py db-health
```
```

```
./deployControlPlan.sh --recover_db
# SSL certificate issues
./scripts/generate_ssl.sh
./scripts/fix_ssl_chain.sh
```

Reset Installation

```bash
# Stop all services
./deployControlPlan.sh stop
# Complete reset (Docker)
docker-compose down -v
docker system prune -f
# Complete reset (Local)
sudo systemctl stop nginx gitea
sudo rm -rf /etc/nginx/sites-enabled/opcp-ai-powered-store
rm -rf softfluid/db/ai_swautomorph.db
# Restart deployment
./deployControlPlan.sh start
```

Debug Mode

```bash
# Enable debug logging
export FLASK_DEBUG=1
export FLASK_ENV=development
# Run with verbose output
./deployControlPlan.sh start locally 2>&1 | tee deployment.log
```